

Sisteme de Operare

Cap 2. Structuri folosite în sisteme de operare

March 6, 2025

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

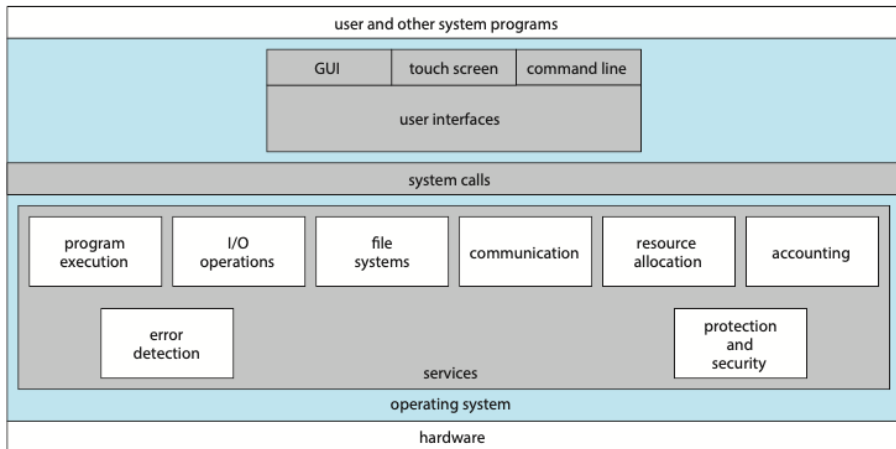
Rolul SO

Putem vedea SO din mai multe puncte de vedere:

- serviciile furnizate de SO;
- interfața pe care o pune la dispoziția utilizatorilor și programatorilor;
- componentele sale și conexiunile cu exteriorul.

În plus, vom considera și cum se furnizează aceste servicii, cum se folosesc și depanează aceste servicii ale SO și care sunt metodologiile de proiectare ale SO.

Clase de servicii oferite de SO (1)



- **interfața utilizator** adesea un sistem de ferestre navigabil cu un mouse; pentru tablete avem interfață touch-screen; opțiunea Command Line Interface (CLI) oferă comenzi text

Clase de servicii oferite de SO (2)

- **execuția programelor** se face prin încărcarea în memorie și rulare; se pot termina normal sau abrupt, indicând eroare
- **operații I/O** se referă la accesarea dispozitivelor, accesarea rețelei sau a unui sistem de fișiere; pentru eficiență și protecție, utilizatorii nu pot accesa direct dispozitivele I/O
- **sistemul de fișiere** servește la crearea, citirea, ștergerea fișierelor, drepturi asupra fișierelor și directoarelor
- **comunicațiile** pot avea loc între procese de pe aceeași mașină sau de pe mașini diferite, interconectate; modalitățile includ **shared memory** respectiv **message passing**
- **detectia erorilor** este necesară în mod constant, pot apărea în CPU sau în memorie, în dispozitivele I/O (parity error on disk, connection failure în rețea, lipsă hârtie la imprimantă), sau în program (overflow aritmetic, accesare invalidă la memorie); SO poate termina programul care generează eroarea, sau chiar opri tot sistemul (system halt)

Servicii pentru funcționarea eficientă a SO

- **alocarea resurselor**: CPU cycles, memoria principală, stocarea fișierelor; vom vedea algoritmi de planificare a proceselor, alocarea imprimantelor, dispozitivelor de stocare USB
- **logging**, ține evidența resurselor utilizate de procese, statistici de utilizare folosite de sysadmini
- **protecție și securitate**, proprietarii informației controlează cine și cum folosește informația; procesele trebuie să își poată citi unul altuia zonele de memorie folosite; securitatea începe cu autentificare în sistem, continuă cu prevenirea accesului neautorizat la dispozitivele I/O precum și a conectorilor de rețea

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

Interpretorul de comenzi

- tratat ca program special; sistemele UNIX și Linux au mai multe **shells**, interpretoare diferite de comenzi, ex. C-shell, Bourne-Again Shell, Korn shell)
- funcția principală este să preia și să execute următoarea comandă utilizator
- în abordarea UNIX, shell-ul nu cunoaște toate comenzile ci folosește fișiere executabile externe (ex. `rm file.txt`, va citi executabilul corespunzător)ș noi comenzi pot fi astfel adăugate SO

Graphical User Interface (GUI)

- comenzile se introduc pe baza unei ferestre, în care user-ul interacționează cu un mouse; oferă o experiență desktop-based
- apărută ca rezultat al cercetărilor de la Xerox PARC, primul GUI a apărut în 1973 pe calculatorul Xerox Alto
- tradițional, sistemele UNIX au fost dominate de CLI; există multe implementări de GUI, ex. K Desktop Environment (KDE), LXDE, XFCE, Gnome etc.
- alegerea GUI-ului este chestiune de preferințe; administratorii de sistem (sysadmin) și **power users** care au cunoștințe de SO folosesc în mod frecvent CLI; CLI este mult mai eficientă, acces mai rapid la unelte; prin GUI avem acces doar la un subset de unelte; activitățile efectuate des pot fi grupate în **shell scripts**, astfel se acestea se pot automatiza ușor
- prin contrast, utilizatorii Windows sunt fericiți să folosească exclusiv GUI (cu productivitate mai scăzută)

System calls (apeluri sistem)

- furnizează o interfață pentru serviciile oferite de SO; disponibile ca funcții apelabile din programe scrise în C un exemplu:
`cp in.txt out.txt`
- se realizează următoarele apeluri sistem:
 - deschiderea fișierului sursă și crearea celui destinație;
 - verificarea existenței sursei, a drepturilor de acces ale sursei și folder-ului destinație;
 - afișarea mesajelor de eroare dacă este cazul, terminarea abruptă;
 - ștergerea destinației, dacă există deja;
 - interogarea utilizatorului, pentru GUI;
 - citirea bloc cu bloc, tratarea situației de EOF respectiv erori de paritate;
 - scrierea va putea eșua din cauza lipsei spațiului pe mediul destinație;
 - la terminare, SO va închide ambele fișiere;
 - eventual se va afișa un mesaj către utilizator;
 - terminarea procesului se face tot printr-un system call.
- în general SO execută mii de apeluri sistem pe secundă

Application Programming Interface (API)

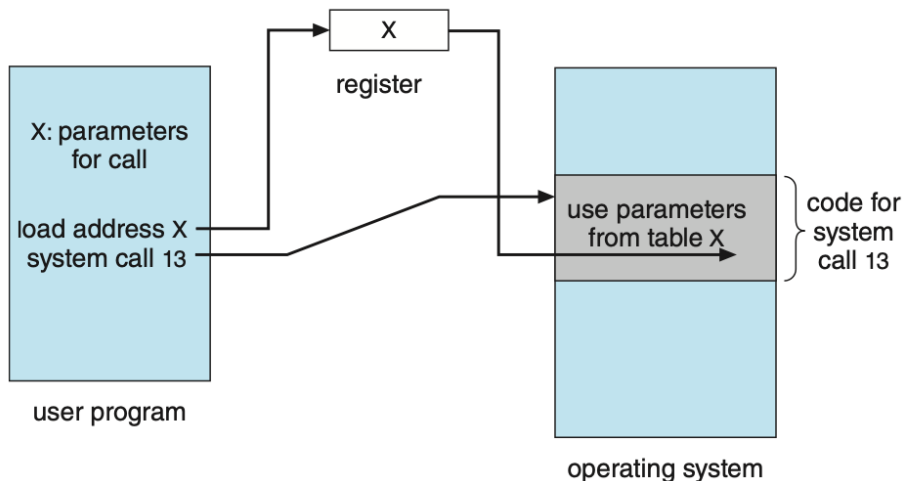
- specifică setul de funcții pe care dezvoltatorul de aplicații le poate folosi
- Windows API, POSIX API (UNIX, Linux, macOS), Java API
- biblioteca **libc** oferă colecția de apeluri sistem, accesibile din programele C
- apelul de sistem pentru citirea din fișier (v. `$ man read`)

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count)
```

- fd - file descriptor, buf - buffer-ul unde sunt aduse datele, count - mărimea maximă a blocului citit
- valoarea întoarsă este numărul de bytes citați, sau -1 în caz de eroare

Transmiterea parametrilor într-un syscall



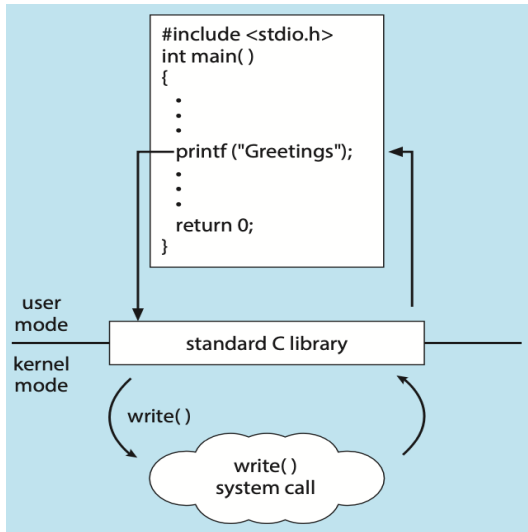
- metode: în registru (prin valoare), prin adresa unei tabele, prin stivă

Exemple de tipuri de apeluri sistem

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



Biblioteca standard C



- furnizează interfața cu apelurile de sistem
- se face comutarea între user-mode și kernel-mode

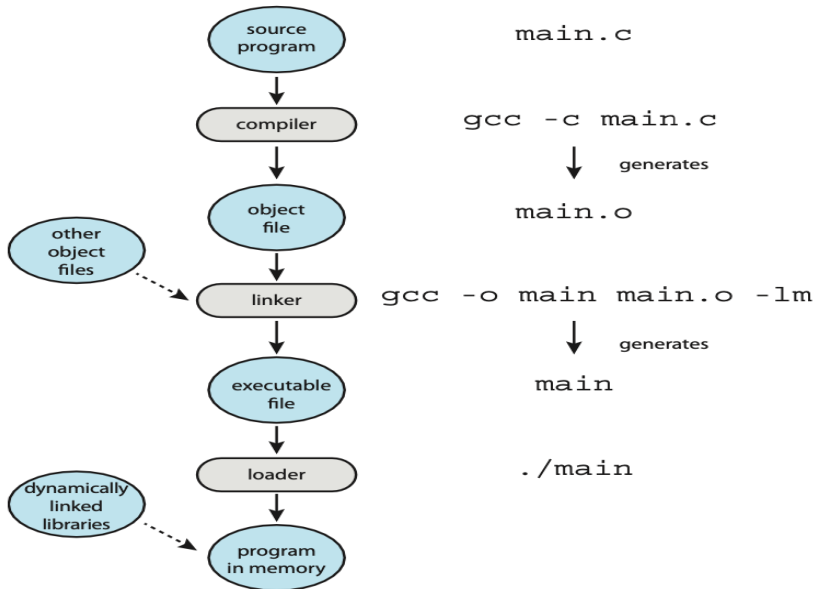
- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 **Servicii sistem**
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

Mediul de dezvoltare și execuție

- **managementul fișierelor:** programe ce creează, copiază, șterg, redenumesc, tipăresc, afișează fișiere și directoare
- **informații de stare:** timpul curent, spațiu de memorie / disc rămas liber, utilizatori activi; performanța sistemului, log-uri, informații de configurare a sistemului (ex. Windows registry)
- **modificarea fișierelor:** editoare de text, transformarea textelor folosind reguli (sed, awk, grep)
- **utilitare pentru limbaje de programare:** compilatoare, asamblatoare, depanatoare, interpretoare (C, C++, Java, Python etc.)
- **încărcarea și execuția programelor:** (relocatable) loaders, linkers
- **comunicații:** programe pentru conexiuni virtuale între procese sau utilizatori (remote desktop, browsers, clienți de e-mail, remote login, remote copy)
- **servicii background:** servicii, subsisteme sau daemons; ex. network daemon care mapează conexiunile incoming către procesele specifice; planificatoare care pornesc procese la data și ora fixată

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders**
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

Rolurile linker-ului și loader-ului



Rolurile linker-ului și loader-ului

- programul executabil `a.out` sau `prog.exe` se află pe disc
- formatul fișierului compilat este **relocabil**, destinat să poată fi încărcat începând cu orice adresă de memorie
- link editor-ul (linker-ul) combină aceste fișiere relocabile într-un fișier executabil; poate adăuga și alte fișiere obiect sau biblioteci
- loader-ul încarcă executabilul binar în memorie și face relocarea codului, ajustează codul să poată rula, setând adresele salturilor absolute din cod
- la execuția `./main` -ului, shell-ul creează un nou proces prin apelul de sistem `fork()`; ulterior invocă loader-ul prin apelul `exec()`, cu argument numele executabilului
- de obicei executabilul nu are nevoie doar de biblioteci **statice**, deja incluse în imaginea executabilului, ci și de biblioteci **dinamice**, existente ca fișiere separate

De ce sunt aplicațiile dependente de SO

- fiecare SO furnizează un set unic de apeluri sistem
- aplicațiile pot fi făcute independente de SO prin:
 - 1 scrierea ei într-un limbaj interpretat (Ruby, Python); performanța scade, se oferă un sub-set limitat de feature-uri SO
 - 2 folosirea unui limbaj pentru o mașină virtuală ce furnizează un Run-Time Environment (RTE), precum JVM; aceasta include un loader, byte-code verifior; aceleași dezavantaje
 - 3 aplicația este dezvoltată într-un limbaj și un API standard; se recompilează pentru fiecare SO; API-ul asigură păstrarea compatibilității (ex. POSIX API)
- probleme:
 - formatul binar al executabilului este diferit pentru fiecare SO
 - CPU-uri diferite au seturi de instrucțiuni diferite
 - apelurile sistem se numesc diferit, au alte structuri, alți parametri

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO**
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

Tipuri de structurare a software-ului SO

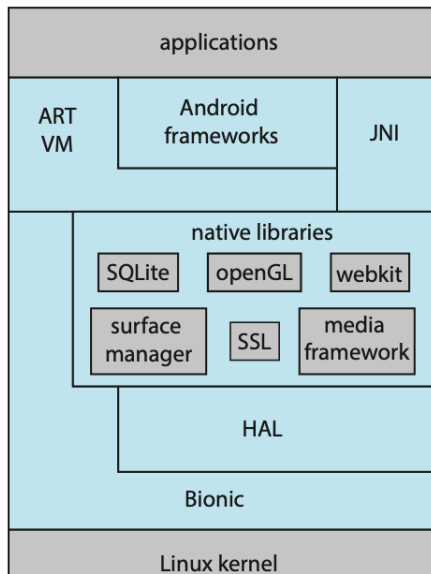
- un SO este un program complex; adesea este împărțit în componente (proceduri) mai mici
- **structura monolitică** a SO consistă într-un kernel scris ca o singură imagine binară care se încarcă și rulează într-un singur spațiu de adrese; dificil de implementat și extins, dar rapide
- SO **stratificate** presupun împărțirea kernel-ului în **module**, fiecare cu propria funcționalitate; sunt mai simplu de construit și depanat, pentru că fiecare nivel folosește doar serviciile nivelelor de nivel inferior
- structura de tip **microkernel** elimină din imaginea kernel-ului toate serviciile neesențiale (sisteme de fișiere, drivere, programe aplicație), tipic rămân doar management-ul memoriei, al CPU-ului și mecanismul IPC¹ (ex. Darwin iOS, QNX Neutrino microkernels)

¹Inter-Process Communication

Module

- **loadable kernel modules** (LKM) este o abordare ce separă kernel-ul de modulele ce încarcă servicii precum sistemul de fișiere, drivere pentru dispozitive; încărcarea modulelor se poate face fie la boot fie în timpul rulării; abordarea este comună sistemelor UNIX (Linux, macOS, Solaris) respectiv Windows
- abordarea modulară este mai avantajoasă: modulele pot fi adăugate dinamic, nu este necesară o recompilare a kernel-ului ca în cazul celor monolitice
- în practică se folosește un mix de abordări: Linux și Windows sunt monolitice, dar au module încărcate dinamic
- macOS este sistemul desktop, iar iOS destinat dispozitivelor mobile; arhitectura lor este stratificată: user experience layer (Aqua / Springboard UIs), application framework layer (API pentru limbajele Objective-C și Swift), core frameworks (Quicktime, OpenGL), kernel (Darwin, bazat pe kernel-ul BSD UNIX)

Android OS



- proiectat de Open Handset Alliance (condus de Google)
- aplicațiile folosesc o versiune de Java API optimizată, bytecode-ul e compilat în cod mașină la instalare
- JNI^a permite accesul direct la hardware (neportabil)
- HAL^b abstractizează camera, senzorul GPS (portabilitate)
- Google a dezvoltat Bionic C library, și a modificat și kernel-ul Linux

^aJava Native Interface

^bHardware Abstract Layer

Windows Subsystem for Linux (WSL)

- folosește o arhitectură hibridă pentru a emula alte SO; subsistemul WSL funcționează în user-mode și solicită servicii kernel-ului Windows
- Windows creează o instanță Linux, adică un proces **init**, care va rula comanda dată; furnizează programului Linux mediul de execuție
- serviciul kernel LXSS furnizează maparea syscall-urilor Linux în echivalentul Windows

user mode

bash.exe

Linux instance

init

/bin/bash

kernel mode

fork()

LXSS/LXCore

CreateProcess()

Windows kernel

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO**
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB

Generarea SO

- generarea presupune: configurarea specifică hardware-ului, compilarea SO din cod sursă, instalarea și bootarea
- se pot aduce sursele de la <http://www.kernel.org>, configura cu `make menuconfig`, compila imaginea de kernel denumită `vmlinuz` cu `make` iar modulele cu `make modules`; instalarea cu `make install`
- procesul de boot are loc în etape: un mic program pornește din memoria non-volatilă **BIOS**, ce încarcă al doilea boot loader din **boot block**-ul discului extern, de la o locație fixă; acesta încarcă kernel-ul
- BIOS-ul este înlocuit azi de UEFI², ce are suport pentru sisteme 64b și medii de stocare de capacitate mare; boot loader într-o singură etapă
- GRUB este un bootloader open-source, configurabil printr-un fișier
- imaginea `vmlinuz` este decomprimată în RAM la boot; se creează filesystem-ul `initramfs`, ce conține driverele și modulele pentru hardware-ul specific; înlocuit după montarea filesystem-ului real; se creează procesul inițial `systemd` ce pornește celelalte servicii

²Unified Extensible Firmware Interface

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO**
- 8 Procesul de build cu Makefiles. GDB

Analiza erorilor

- procesul de **debugging** (depanare) presupune căutarea și repararea erorilor; include și **performance tuning**, optimizarea performanței sistemului, prin eliminarea **bottleneck**-urilor (puncte critice)
- unui proces crăpat (failed) i se poate inspecta **log file**-ul și găsi problema survenită; SO poate de asemenea intercepta crăparea și salva un **core dump**, imaginea memoriei procesului, ce se poate analiza ulterior cu un program de depanare **debugger** (ex. GDB)
- un failure în kernel este denumit **crash**, și se salvează un **crash dump**

Monitorizarea performanțelor SO

- are ca scop identificarea bottleneck-urilor; unelte pot identifica probleme:
 - per process:
 - `ps` raportează informația pentru un proces sau mai multe
 - `top` statistici în timp real pentru procesele curente
 - pentru întreg sistemul:
 - `vmstat` statistica utilizării memoriei
 - `netstat` statistici pentru utilizarea interfeței de rețea
 - `iostat` statistici pentru utilizarea dispozitivelor I/O³
- majoritatea acestor unelte citesc informație din filesystem-ul `/proc`, pseudo-filesystem ce există doar în memoria kernel-ului; organizat în directoare, fiecare director este de fapt un identificator de proces
- Windows furnizează Task Manager, ce include informații despre procese și aplicații, statistici de folosire a memoriei și a CPU, precum și a interfețelor de rețea

³ posibil să aveți nevoie să instalați pachetul `sysstat` cu "`sudo dnf install sysstat`"

Tracing

- realizează listarea tuturor apelurilor de sistem invocate de un proces
- uneltele Linux pentru tracing-ul evenimentelor:
 - per process:
 - **strace**⁴ relevă apelurile sistem invocate de un proces
 - **gdb** este depanatorul de programe la nivel de cod sursă, GNU Debugger
 - sistem:
 - **perf** este o colecție de unelte Linux pentru evaluarea performanțelor sistemului
 - **tcpdump** colectează pachete de rețea
 - **bcc**, BFP Compiler Collection este un toolkit pentru tracing-ul sistemelor Linux, urmărirea și depanarea kernel-ului în acțiune

Kernighan's Law: "Debugging is twice as hard as writting code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."⁵

⁴toate aceste unelte trebuie instalate cu "dnf install"

⁵<https://github.com/dwmkerr/hacker-laws>

Exemplu de strace

```
$ man touch
```

```
touch - change file timestamps
```

```
touch [OPTION]... FILE...
```

```
A FILE argument that does not exist is created empty, ...
```

```
$ strace touch a.txt
```

```
execve("/usr/bin/touch", ["touch", "a.txt"], 0xffebc348) = 0
```

```
...
```

```
openat(AT_FDCWD, "a.txt", O_WRONLY|O_CREAT|O_NOCTTY|
                                O_NONBLOCK, 0666) = 3
```

```
dup3(3, 0, 0) = 0
```

```
close(3) = 0
```

```
utimensat(0, NULL, NULL, 0) = 0
```

```
close(0) = 0
```

```
close(1) = 0
```

```
close(2) = 0
```

- 1 Serviciile asigurate de SO
- 2 Interfața utilizator a SO
- 3 Servicii sistem
- 4 Linkers și loaders
- 5 Structura SO
- 6 Compilarea și bootarea SO
- 7 Depanarea SO
- 8 Procesul de build cu Makefiles. GDB**

Structura proiectului (1)

- fișierul **func.c**, funcția `cmmdc()` scrisă separat de main:

```
#include "func.h"
```

```
int cmmdc(int a, int b) {
    if (b == 0)
        return a;

    return cmmdc(b, a % b);
}
```

- fișierul **func.h** denumit header declară funcția, pentru a putea genera cod care o cheamă:

```
int cmmdc(int a, int b);
```

Structura proiectului (2)

- fișierul **main.c**, funcția main():

```
#include <stdio.h>
#include <stdlib.h>
#include "func.h"
int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "prea putini parametri\n");
        exit(1); }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    if (a < b) {
        fprintf(stderr, "ordine incorecta\n");
        exit(2); }
    printf("a: %d b: %d cmmdc: %d\n", a, b, cmmdc(a, b));
    return 0;
}
```

Apeluri de sistem și funcții de bibliotecă

- **directivele include** sunt de două feluri:
 - cele cu paranteze unghiulare vor căuta fișierul header în folder-ele sistem, și specificate în căi de căutare
 - cele cu ghilimele vor începe căutarea cu folder-ul curent, și o vor încheia cu folder-ele sistem
- parametrii lui `main()` vin din linia de comandă; sunt 3 parametri, primul - `argv[0]` - este **numele executabilului**
- apelul de sistem `exit()` termină programul cu codul de eroare specificat
- `atoi()` este o funcție din standard C library, care poate eșua⁶
- funcția `fprintf()` tipărește rezultatul la stream-ul de ieșire; orice proces Linux are 3 fișiere deschise implicit, standard input cu file descriptor-ul 0, standard output (1) și standard error (2)
- biblioteca standard C este legată implicit la compilare

⁶vedeți pagina de manual din secțiunea 3 pentru toate aceste funcții, ex. "man 3 atoi"

Fișiere makefile⁷

- este o "rețetă" care dictează cum se compilează și link-editează programul
- un program este format din **fișiere sursă** care conțin codul funcțiilor și **fișiere header** care, în general, doar declară antetele funcțiilor (semnăturile) folosite într-un alt fișier sursă, pentru ca acea funcție să știe cum să cheme funcția
- fișierul Makefile este o succesiune de reguli despărțite de o linie vidă
- o regulă specifică:
 - pe prima linie numele, caracterul ":" și fișierele de care depinde execuția regulii;
 - următoarele linii conțin comanda care se execută.
- compilatorul folosit este GNU C Compiler (GCC), cu mai multe opțiuni (-o: specifică numele output-ului, -g: adaugă în cod informații de depanare, -Wall: activează toate warning-urile, -c: face doar compilare, nu și link-editare)

⁷https://www.cs.swarthmore.edu/~newhall/unixhelp/howto_makefiles.html

Fișier Makefile simplu

```
all: func.h func.c main.c
    gcc -g -Wall -o cmmdc func.c main.c

clean:
    rm -f cmmdc
```

- execuția lansează prima regulă:

```
[user@fedora extra]$ make
gcc -g -Wall -o cmmdc func.c main.c
[user@fedora extra]$ ./cmmdc 273 26
a: 273 b: 26 cmmdc: 13
[user@fedora extra]$
```

Fișierul Makefile2 (1)

```
# compilatorul
CC = gcc

# optiuni de compilare
# -g      adauga informatii de depanare executabilului generat
# -Wall   activeaza majoritatea avertismentelor compilatorului
CFLAGS = -g -Wall

# executabilul generat
TARGET = cmmdc

# la invocarea simpla, se cheama prima regula
all: $(TARGET)
```


Fișierul Makefile2 (2)

```
$(TARGET): func.o main.o
    $(CC) $(CFLAGS) -o $(TARGET) func.o main.o

func.o: func.h func.c
    $(CC) $(CFLAGS) -c func.c

main.o: func.h main.c
    $(CC) $(CFLAGS) -c main.c

clean:
    $(RM) $(TARGET) *.o
```

Fișierul Makefile3

```
CC      = gcc
CFLAGS = -g -Wall
TARGET = cmmdc
SRCS    = func.c main.c
OBJS    = $(SRCS:.c=.o)      # suffix replacement intr-un macro

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJS)

# suffix replace, face .o din .c, $< - sursa, $@ - destinatia
.c.o:
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    $(RM) $(TARGET) *.o
```

Depanarea cu GDB (1)

```
[user@fedora extra]$ gdb ./cmmdc
Reading symbols from ./cmmdc...
(gdb) set args 273 26
(gdb) info sources
/home/user/extra/cmmdc:
/home/user/extra/main.c, /home/user/extra/func.c
(gdb) l
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "func.h"
4
5 int main(int argc, char *argv[]) {
6     if (argc < 3) {
7         fprintf(stderr, "prea putini parametri\n");
8         exit(1);
9     } }
```

Depanarea cu GDB (2)

```
(gdb) l func.c:cmmdc
```

```
1 #include "func.h"
```

```
2
```

```
3 int cmmdc(int a, int b) {
```

```
4     if (b == 0)
```

```
5         return a;
```

```
6
```

```
7     return cmmdc(b, a % b);
```

```
8 }
```

```
(gdb) br func.c:4
```

```
Breakpoint 1 at 0x4007a8: file func.c, line 4.
```

```
(gdb) r
```

```
Starting program: /home/user/extra/cmmdc 273 26
```

Depanarea cu GDB (3)

Breakpoint 1, cmmdc (a=273, b=26) at func.c:4

```
4  if (b == 0)
```

```
(gdb) bt
```

```
#0  cmmdc (a=273, b=26) at func.c:4
```

```
#1  0x004008a0 in main (argc=3, argv=0xfffff338) at main.c:18
```

```
(gdb) c
```

Continuing.

Breakpoint 1, cmmdc (a=26, b=13) at func.c:4

```
4  if (b == 0)
```

```
(gdb) c
```

Continuing.

Depanarea cu GDB (4)

```
Breakpoint 1, cmmdc (a=13, b=0) at func.c:4
```

```
4   if (b == 0)
```

```
(gdb) p a
```

```
$1 = 13
```

```
(gdb) c
```

```
Continuing.
```

```
a: 273 b: 26 cmmdc: 13
```

```
[Inferior 1 (process 2064) exited normally]
```

```
(gdb) q
```

```
[user@fedora extra]$
```